Energy Price Forecaster: Technical Documentation

Project Abstract: The Energy Price Forecaster is a Python-based machine learning solution designed to predict day-ahead electricity prices for the Italian energy market. Leveraging state-of-the-art models like NHITS and XGBoost, the tool automates the entire forecasting pipeline from data ingestion and feature engineering to model training and evaluation. The primary goal is to provide accurate, multi-hour-ahead price predictions (up to 7 days) to support energy trading, resource management, and strategic decision-making.

This document provides a comprehensive overview of the `Price_forecaster.py` script, a tool designed for forecasting energy prices. It details the script's architecture, workflow key components, models used, and usage instructions.

1. Overview

The purpose of this script is to **predict future energy prices** for the Italian energy market. It implements a complete machine learning pipeline, including data loading, preprocessing, feature engineering, model training, evaluation, and prediction.

The system is flexible, allowing the user to train and evaluate one or more of the following advanced, state of the art forecasting models:

- **XGBoost**: A powerful gradient-boosting model configured for multi-output regression.
- **NBEATS** (Neural Basis Expansion Analysis for Time Series): A deep learning model designed to capture complex time-series patterns like trends and seasonality.
- **NHITS** (Neural Hierarchical Interpolation for Time Series): An evolution of NBEATS, optimized for long-horizon forecasting with improved performance and efficiency.

By default, the script is configured to train the **NHITS** model to forecast prices **7 days (168 hours)** into the future, and also univariate (just price variable).

---

2. Setup and Prerequisites

Before running the script, you must prepare the environment and download the necessary datasets.

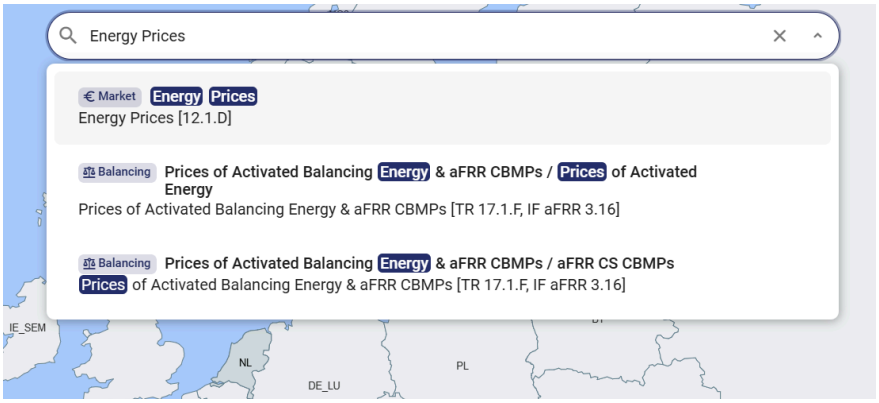
Step 1: Obtain Datasets You need to place two specific files into the `datasets_folder`:

1. `Estrazione consumi orari Innothon.xlsx`: This is the energy consumption dataset.
2. `GUI_ENERGY_PRICES_202401010000-202501010000.csv` and `GUI_ENERGY_PRICES_202501010000-202601010000.csv`: This is the historical energy price data.

Source Citation: European Network of Transmission System Operators for Electricity (ENTSO-E). (2025). *Day-ahead Prices for Italy Central South (IT-CSOUTH) Area*. Retrieved October 12, 2025, from https://newtransparency.entsoe.eu/

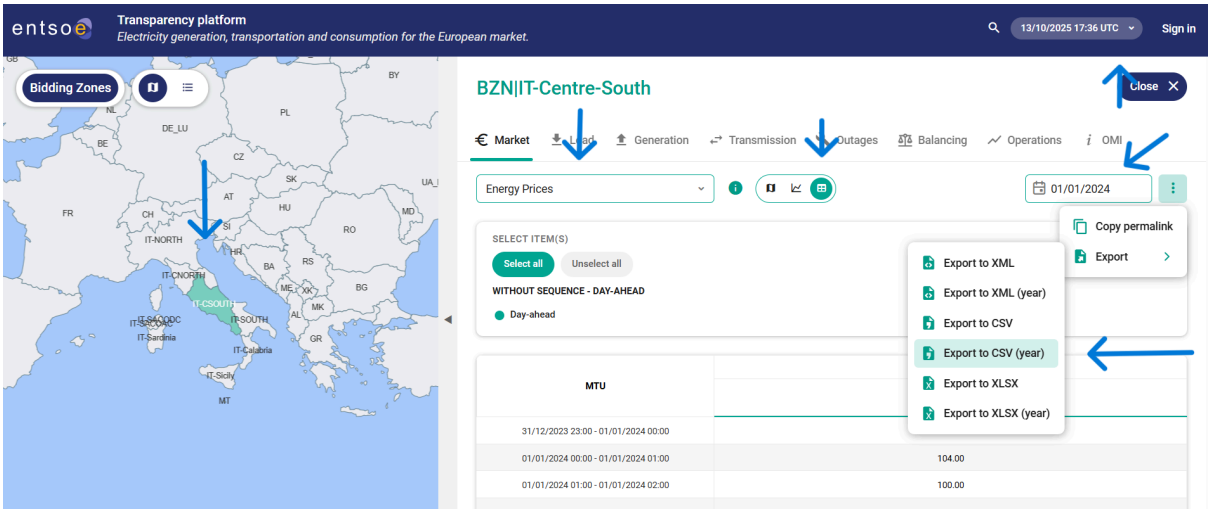
Instructions: It can be downloaded from the ENTSO-E Transparency Platform following these steps:

- Go to the ENTSO-E Transparency Platform.
- In the search bar, type "Energy Prices".



Then from left to right:

- On the map, click on the **Italy Central South region ("IT-CSOUTH")**.
- On the right panel, click the **"Tabular view of data"** icon (where are the 3 icons, the right one).
- Leave the settings to UCT.
- Set the date range to start from January 1, 2025.
- Click the three dots (...) next to the date, select **Export**, and download the **CSV (Year)**. You may need to sign in to download the file.



    - Place the downloaded CSV called in the `datasets_folder`. If they are not called as above, rename as they are stated, in order to make the program work correctly.

In order to make the process faster and easier, an automatic script can be created in the future for getting this data using python, but it's required to send an email to the ENTSO-E Platform email (procedure described on their site) and receive the token in order to make it work. It can be done subsequently.

Step 2: Environment Setup Click on the `Create_environment.bat` file (on Windows) or the equivalent `.sh` script (on Linux/macOS), using run `./forecast_setup.sh`. This will install all the required Python libraries.

If you already have python, It'll ask if you want to install the latest python stable release, otherwise it will automatically download it. After downloading it will open a new window, to confirm the installation of python. Confirm it.

Then it will proceed to install all the required python libraries. If a GPU with CUDA is detected, it'll ask if you want to install CUDA libraries.

It'll also ask if you want to run the script from the cmd. If everything is set correctly, it should start running. Otherwise you can try the next step.

Step 3: Run the script When the environment is completely set up, run the python code with visual studio code, or opening a command line in this folder, then write

call env\Scripts\activate

python code\Price_forecaster.py

---

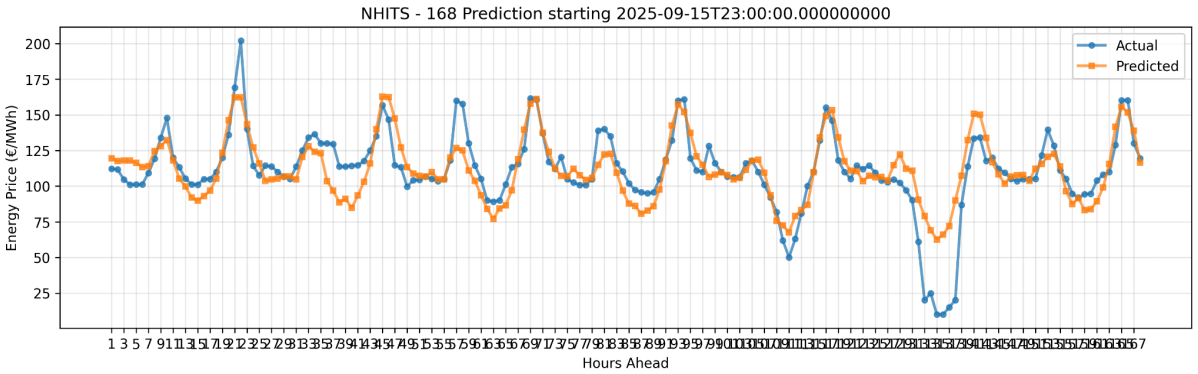
3. Code Architecture and Workflow

The script is structured into several classes, each handling a specific part of the ML pipeline. The overall workflow is as follows:

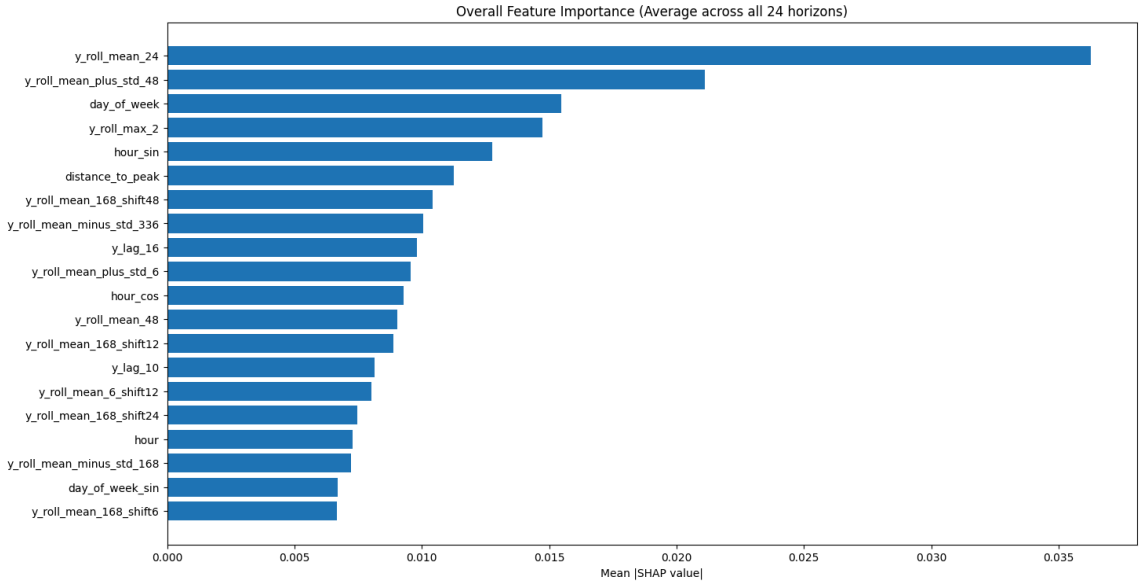
1. **Configuration**: Global parameters like the model to run (`models_to_run`), forecast horizon (`n_out`), and model-specific hyperparameters are defined at the top of the script.
2. **Data Ingestion & Preprocessing (`CreateDatasets` & `DataLoaderClass`)**:
    - The script first checks if the required datasets (`Prices_2025.csv`, `Consumi_Orari_2024.csv`, `Weather_2004_2025.csv`) exist.

    - If a dataset is missing, it automatically generates it:
        - **Price Data**: Raw price data is loaded, cleaned, and resampled from 15-minute to hourly intervals.
        - **Consumption Data**: Consumption data from the Excel file is cleaned, interpolated to handle missing values, and transformed into a usable format.
        - **Weather Data**: Historical weather data for Milan, Rome, and Naples is fetched from the Open-Meteo API.
    - The processed datasets are then loaded into memory.
3. **Data Integration (`DataIntegrator`)**: The individual price, consumption, and weather datasets are merged into a single, comprehensive DataFrame based on their timestamps.
4. **Feature Engineering (`FeatureEngineer`)**: This is a critical step where the script creates new features to improve model accuracy. These include:
    - **Temporal Features**: Hour of the day, day of the week, holidays, and cyclical features (e.g., `hour_sin`, `hour_cos`).
    - **Lagged Features**: Past values of the energy price (e.g., price 24 hours ago).
    - **Rolling Statistics**: Rolling averages, standard deviations, min/max values over various time windows to capture local trends and volatility.
    - **Weather & Consumption Features**: Derived metrics from weather (e.g., Cooling Degree Days) and consumption data.

5. **Plotting with confidence levels(`PredictionIntervals and functions`):** The model is later evaluated using **Model Training and Prediction (`PriceForecaster` and Model Classes)**:
    - The data is split into training, validation, and test sets.
    - Features and target values are normalized using `StandardScaler`.
    - The data is reshaped into sequences suitable for the chosen model.
    - The selected model (e.g., NHITS) is trained on the data. An **Early Stopping** mechanism is used to prevent overfitting.
    - The trained model, along with the data scalers, is saved to the `models` directory.
6. **Evaluation and Visualization**:
    - The model's performance is evaluated on the test set using metrics like **MAE** (Mean Absolute Error), **RMSE** (Root Mean Squared Error), and **MAPE** (Mean Absolute Percentage Error).
    - Several plots are generated to visualize the model's predictions against the actual values, providing insights into its performance across different time horizons. Plots with confidence levels, are later used when loading the model, to show how well it performs on the test dataset.



- When using the XGBoost Model, it will even show SHAP values charts. They are used to measure how much the parameters used are important for calculating specific hours price values or overall values. It's a very useful Explainable AI tool. This paper has been used as a reference.



---

4. Model Architectures, Behavior and Results

The script allows for the use of three distinct models, each with a different approach to time series forecasting. By clicking on the links you'll be directed to the related papers that delve deep into the models functioning. They all have been trained and tested mainly using datasets prices of 2024 and 2025, trying both with and without the use of the weather and the consumption datasets, and trying by implementing multiple additional features. Results shown are some of the best captured during the parameters choice.

1. XGBoost (eXtreme Gradient Boosting)

Structure: XGBoost is not a neural network but an **ensemble learning method** based on decision trees. It builds a "forest" of decision trees one after the other. Each new tree is trained to correct the errors made by the previous ones.

In this script, it's configured as a **multi-output regressor** (`'num_target': n_out`). This means a single XGBoost model is trained to predict all future time steps (e.g., all 168 hours of the next week) simultaneously, rather than training 168 separate models.

Behavior:

- **Feature-Driven:** XGBoost does not inherently understand the sequence or "flow" of time. It works with the **features** you provide. It learns complex, non-linear relationships between the input features (like price 24 hours ago, day of the week, rolling averages, weather data) and the future price at each specific hour.
- **How it Predicts:** To predict the price for `Hour 1`, `Hour 2`, ..., `Hour 168`, it uses the current set of features to make 168 independent predictions at once. For example, it might learn that "if `is_weekend` is 1 and `y_lag_24` (price 24h ago) is low, then the price in 12 hours will likely be low."

Results: Based on the 2024 energy_price, using a great number of features (250, using the add_price_features() function), and parameters:

{'num_target': 168, 'n_estimators': 200, 'learning_rate': 0.1, 'max_depth': 8, 'gamma': 0.1, 'subsample': 0.9, 'colsample_bytree': 0.4, 'colsample_bylevel': 0.8, 'colsample_bynode': 0.8, 'lambda': 1.6, 'alpha': 0.2, 'random_state': 42, 'tree_method': 'gpu_hist', 'predictor': 'gpu_predictor'}

with loss metrics:

```
=== Experiment summary ===
XGBoost: {'MAE': '23.6230', 'RMSE': '32.8965', 'MAPE': '21.06'}
```

---

2. NBEATS (Neural Basis Expansion Analysis for Time Series)

Structure: NBEATS is a deep neural network designed specifically for time series forecasting without needing manual feature engineering, working well as a univariate model. Its architecture is based on a **residual learning** principle, organized into "stacks" and "blocks."

- **Stacks:** The model is divided into stacks, each specialized for a certain task. The script configures it with `['trend', 'seasonality', 'generic']` stacks.
- **Blocks:** Each stack contains several blocks. A block takes a piece of the time series, makes a forecast, and also models the input it saw (a `backcast`).

- **Residual Learning:** The `backcast` is subtracted from the original input, and the leftover signal (the residual) is passed to the next block. This allows each subsequent block to focus on modeling the patterns the previous ones couldn't capture.

Behavior:

- **Decomposition:** NBEATS works by breaking down the time series into simpler components.
    1. The **Trend Stack** first tries to model the long-term trend of the series.
    2. This trend is then removed, and the **Seasonality Stack** models the recurring weekly or daily patterns from what's left.
    3. Finally, this seasonality is removed, and the **Generic Stack** models any other complex patterns that remain.
- **Final Forecast:** The final prediction is the sum of the forecasts from all the blocks in all the stacks.

Results: Based on the 2024 energy_price as a univariate model dataset with parameters:

{'n_in': 168, 'n_out': 168, 'step': 1, 'n_blocks': 4, 'n_layers': 2, 'layer_width': 512, 'dropout_rate': 0.1, 'learning_rate': 3e-05, 'seasonality_period': 168, 'n_harmonics': 12, 'theta_dim': 12, 'stack_types': ['trend', 'seasonality', 'generic'], 'epochs': 50, 'batch_size': 16, 'n_features': 1}

with loss metrics:

```
=== Experiment summary ===
NBEATS: {'MAE': '29.8903', 'RMSE': '38.6880', 'MAPE': '25.38'}
```

---

3. NHITS (Neural Hierarchical Interpolation for Time Series)

Structure: NHITS is an evolution of NBEATS, sharing the same block-and-stack architecture with residual learning. Its key innovation is **hierarchical interpolation** and **multi-rate signal processing**.

- **Pooling Layers:** Before processing the time series, each stack in NHITS first downsamples the input using a pooling layer (`nn.MaxPool1d`).
- **Multiple Resolutions:** Different stacks have different pooling sizes (`pool_kernel_sizes`). This means some stacks look at a smoothed, low-resolution version of the time series, while others look at a high-resolution version.

Behavior:

- **Learning at Different Scales:** This hierarchical structure allows the model to learn patterns at various frequencies simultaneously.

1. Stacks with large pooling sizes learn the **low-frequency signals** (the slow-moving, long-term trend).
2. Stacks with smaller pooling sizes learn **higher-frequency signals** (like weekly or daily seasonality).

- **Efficiency and Accuracy:** By first identifying and removing the main trend, the model can then focus more effectively on the subtler, shorter-term patterns in the residuals. This makes it more efficient and often more accurate than NBEATS, especially for long-horizon forecasting.

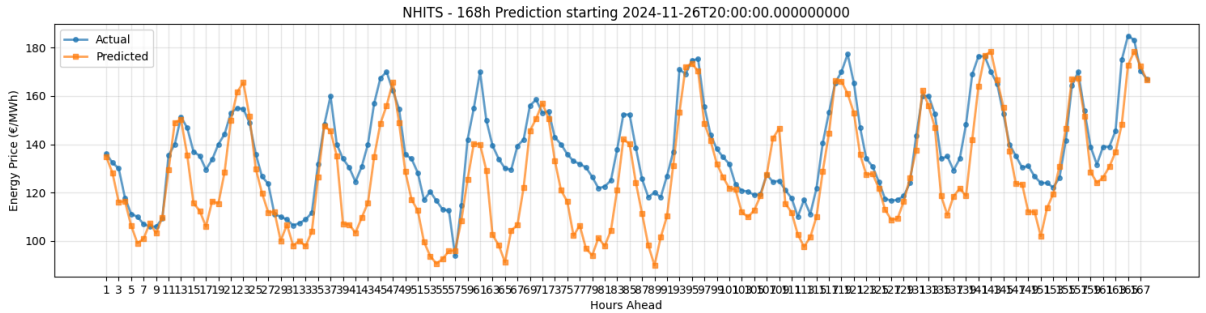
Results: Based on the 2024 energy_price as a univariate model dataset with parameters:

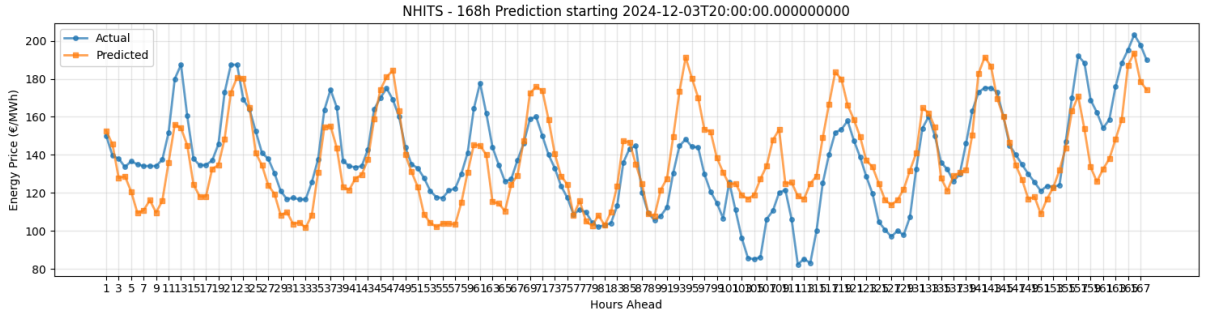
{'n_in': 168, 'n_out': 168, 'step': 1, 'n_blocks': [5, 3, 3, 2, 2, 2, 2], 'mlp_units': [512], 'pool_kernel_sizes': [168, 48, 24, 12, 4, 2, 1], 'n_freq_downsample': [48, 24, 12, 4, 2, 1, 1], 'dropout_rate': 0.1, 'learning_rate': 3e-05, 'epochs': 100, 'batch_size': 16, 'activation': 'relu', 'n_features': 1}

with loss metrics:

```
=== Experiment summary ===
NHITS: {'MAE': '22.1674', 'RMSE': '29.2909', 'MAPE': '20.49'}
```

Conclusions: We've observed that using the NHITS model gave the best results over multiple parameters choices, and by using only the price value, it also ended up being very lightweight in terms of memory used. It also revealed to be the one that best captures peak values.





---

5. How to Use the Script

The script is designed to be run from top to bottom and can be customized easily.

Training a Model

To choose which model to train, modify the `models_to_run` list at the top of the file. For example:

- To train only NHITS: `models_to_run=['NHITS']`
- To train all three models: `models_to_run=['NHITS', 'NBEATS', 'XGBoost']`

After setting the desired models, simply run the script. It will handle the entire training and evaluation pipeline automatically and save the trained models.

Parameters

On top of the code there are some global values that can be modified in order to get different behaviours from the code like:

- how many hours ahead are being predicted (n_out set to 168, one week)
- parameters choice for each model
- number of windows to plot (helping to visualize how well the model behaves with the test set)
- size of train, validation and test sets
- dataset to merge (weather and consumption). Currently set to false. The NHITS and NBEAST work pretty well with just the univariate sequences, and the model results being pretty lightweight in this way, but it's possible to add more features.
- custom features to add. It's advisable to set them True when using XGBoost, it works better with features, especially custom price features.

Making Predictions with a Trained Model

The `run()` function at the end of the script demonstrates the complete workflow. After training, it automatically initializes a new `PriceForecaster` instance and calls the `load_and_predict` function. This function:

1. Loads the most recently trained model (in this case, NHITS) and its associated scalers.
2. Prepares the latest available data.
3. Makes a forecast for the next `n_out` hours (e.g., 168 hours).
4. Prints the final prediction to the console.

The final output will be a printout showing the predicted energy prices for the next 7 days, starting from the last timestamp in the dataset.